ML Experiment Number 10

Student Name : Pranay Mestry

Batch : CS 2

Roll Number : CS 31

Aim

To implement DBSCAN clustering using Python for customer segmentation on the Mall Customer Segmentation dataset and identify arbitrary-shaped clusters along with spatial outliers (noise).

Objectives

1. To understand the principles of Density-Based Spatial Clustering of Applications with Noise (DBSCAN).
2. To implement DBSCAN clustering using Python.3. To preprocess and feature-scale tabular customer data using StandardScaler.
3. To determine optimal hyperparameters (ε and $min_samples$) using k-distance graph analysis.
4. To identify core points, border points, and spatial outliers (noise points denoted by label -1).
5. To evaluate clustering performance using metrics like Silhouette Score.

Relevant Theory

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN is an unsupervised non-parametric clustering algorithm that groups data points based on local density estimations. Unlike centroid-based algorithms (e.g., K-Means), DBSCAN can discover arbitrary-shaped clusters and automatically isolate noisy data points without requiring the user to specify the total number of clusters in advance.

Core Concepts and Parameters:

1. ε (Epsilon): The maximum radius of the neighborhood around a given data point x.
2. $min_samples$ (MinPts): The minimum number of points within the ε-neighborhood required to form a dense region.

Point ClassificationsCore Point:

- A point that has at least $min_samples$ within its ε-neighborhood.
- Border Point: A point that is within the ε-neighborhood of a Core Point but has fewer than $min_samples$ in its own neighborhood.
- Noise Point (Outlier): A point that is neither a Core Point nor a Border Point. Noise points are assigned a cluster label of -1.

Dataset Information

Get the Mall Customer Segmentation dataset from Kaggle using the following link: https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

The dataset contains spending patterns and demographic attributes of mall customers:

- CustomerID: Unique identifier for each customer.
- Gender: Male or Female.
- Age: Age of the customer.
- Annual Income (k$): Annual income of the customer in thousands of dollars.
- Spending Score (1-100): Score assigned based on customer behavior and purchasing patterns.

For clustering, we will use: Annual Income (k$) and Spending Score (1-100).

Task 1: Import Libraries and Load Dataset

- Load essential libraries (numpy, pandas, matplotlib, seaborn, sklearn).

- Load the Mall Customer Segmentation dataset (Mall_Customers.csv).

In [2]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

# Download the dataset if it doesn't exist
if not os.path.exists("Mall_Customers.csv"):
    print("Downloading Mall_Customers.csv...")
    import urllib.request
    url = "https://raw.githubusercontent.com/tirthajyoti/Machine-Learning
    urllib.request.urlretrieve(url, "Mall_Customers.csv")
    print("Download complete.")

# Load the dataset
df = pd.read_csv("Mall_Customers.csv")

# Display the first five rows
print("First five rows of the dataset:")
display(df.head())

# Display dataset information
print("\nDataset Information:")
df.info()

# Check for missing values
print("\nMissing values:")
print(df.isnull().sum())

# Display dataset dimensions
print("\nDataset shape:", df.shape)
```

```
Downloading Mall_Customers.csv...
Download complete.
First five rows of the dataset:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB

Missing values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

Dataset shape: (200, 5)
```

Task 2: Feature Selection and Standardization

- Select target features (Annual Income (k) and Spending Score (1-100)).
- Scale features using Standard Scaler to bring feature dimensions to a comparable range ($\mu=0, \sigma=1$), which is crucial for distance-based neighborhood computations.

In [3]:
```python
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert scaled data to a DataFrame for visualization
X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=['Annual Income (Scaled)', 'Spending Score (Scaled)']
)

print("First five rows of scaled features:")
display(X_scaled_df.head())

# Visualize the standardized data
```

```
plt.figure(figsize=(8, 5))
sns.scatterplot(
    x=X_scaled_df['Annual Income (Scaled)'],
    y=X_scaled_df['Spending Score (Scaled)']
)
plt.title("Standardized Customer Data")
plt.xlabel("Annual Income (Scaled)")
plt.ylabel("Spending Score (Scaled)")
plt.grid(True, alpha=0.3)
plt.show()
```

First five rows of scaled features:

	Annual Income (Scaled)	Spending Score (Scaled)
0	-1.738999	-0.434801
1	-1.738999	1.195704
2	-1.700830	-1.715913
3	-1.700830	1.040418
4	-1.662660	-0.395980



Task 3: Hyperparameter Selection via k-Distance Graph

- Compute Nearest Neighbors using NearestNeighbors ($k = min_samples$).
- Plot sorted k-distances to locate the "elbow point," which indicates the optimal value for ε (Epsilon).

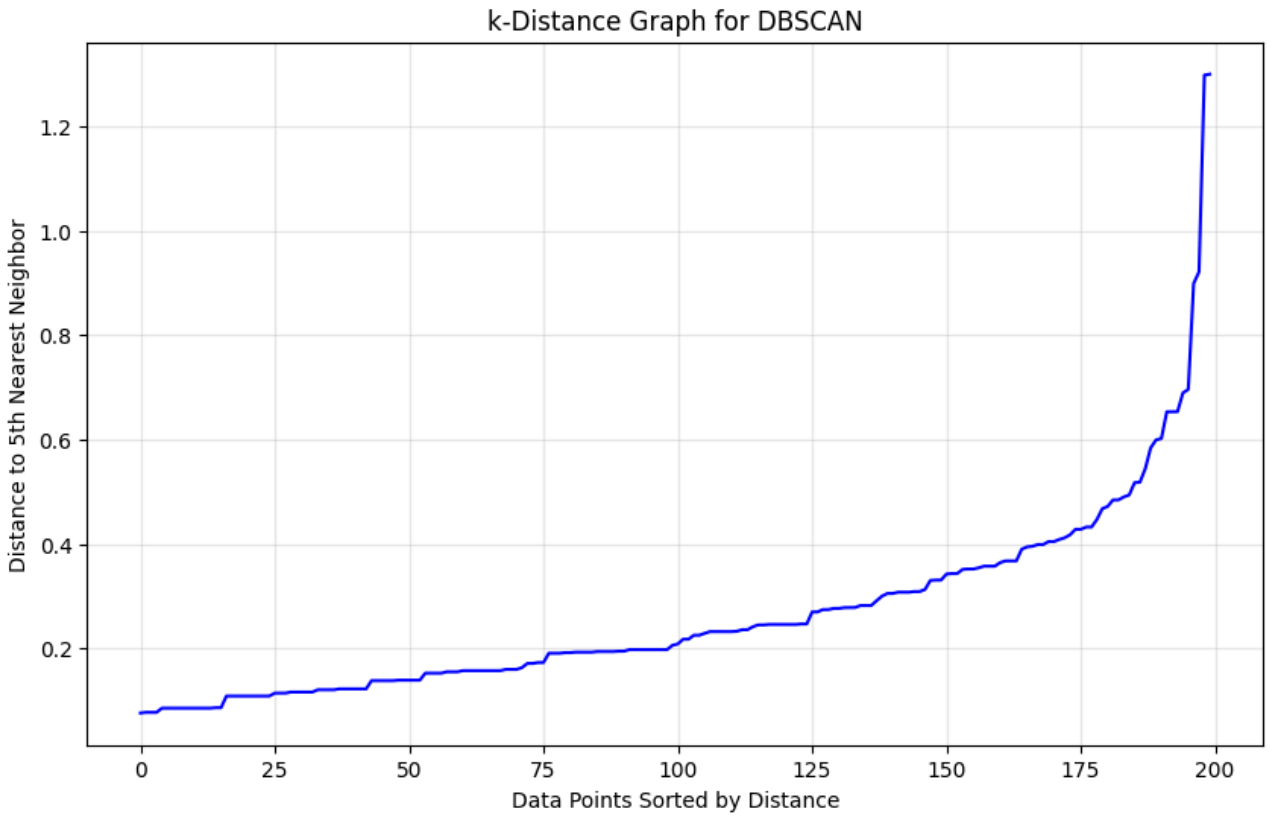
In [4]:
```
min_samples = 5

# Fit the nearest neighbors model
neighbors = NearestNeighbors(n_neighbors=min_samples)
neighbors.fit(X_scaled)

# Calculate distances to nearest neighbors
distances, indices = neighbors.kneighbors(X_scaled)

# Extract distances to the kth nearest neighbor
k_distances = np.sort(distances[:, min_samples - 1])

# Plot the k-distance graph
plt.figure(figsize=(10, 6))
plt.plot(k_distances, color='blue')
plt.xlabel("Data Points Sorted by Distance")
plt.ylabel(f"Distance to {min_samples}th Nearest Neighbor")
plt.title("k-Distance Graph for DBSCAN")
plt.grid(True, alpha=0.3)
plt.show()
```



Task 4: Implement DBSCAN Clustering

- Instantiate sklearn.cluster.DBSCAN with the identified ε and $min_samples$.
- Fit the model on scaled data to generate cluster assignments and noise labels (-1).

In [5]:
```
# Set DBSCAN parameters
eps_value = 0.3
min_samples = 5

# Create and fit the DBSCAN model
dbscan = DBSCAN(
    eps=eps_value,
    min_samples=min_samples
)

labels = dbscan.fit_predict(X_scaled)

# Append cluster labels to the original dataset
df['Cluster'] = labels
```

```
print("Cluster labels:")
print(np.unique(labels))

print("\nFirst five rows with cluster labels:")
display(df.head())
```

```
Cluster labels:
[-1  0  1  2  3  4  5  6]

First five rows with cluster labels:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	2
1	2	Male	21	15	81	0
2	3	Female	20	16	6	1
3	4	Female	23	16	77	0
4	5	Female	31	17	40	2

Task 5: Evaluate DBSCAN Clustering

- Cluster Count: Calculate the number of clusters generated (excluding noise points).
- Noise Point Analysis: Count and evaluate identified spatial outliers (label -1).
- Silhouette Score: Calculate Silhouette Score to evaluate cluster cohesion and separation.

In [6]:
```
unique_labels = set(labels)
clusters = unique_labels - {-1}

# Calculate cluster and noise counts
n_clusters = len(clusters)
n_noise = np.sum(labels == -1)
n_total = len(labels)

print("Number of clusters:", n_clusters)
print("Number of noise points:", n_noise)
print("Total number of customers:", n_total)
print("Noise percentage:", round(n_noise / n_total * 100, 2), "%")

# Identify core points
core_indices = dbscan.core_sample_indices_
n_core = len(core_indices)

print("Number of core points:", n_core)
```

```
# Calculate Silhouette Score when valid
non_noise_mask = labels != -1
non_noise_labels = labels[non_noise_mask]
non_noise_data = X_scaled[non_noise_mask]

if (
    len(np.unique(non_noise_labels)) >= 2
    and len(np.unique(non_noise_labels)) < len(non_noise_labels)
):
    score = silhouette_score(
        non_noise_data,
        non_noise_labels
    )
    print("Silhouette Score (excluding noise):", round(score, 4))
else:
    print("Silhouette Score cannot be calculated reliably: "
          "at least two non-noise clusters are required.")
```

```
Number of clusters: 7
Number of noise points: 35
Total number of customers: 200
Noise percentage: 17.5 %
Number of core points: 138
Silhouette Score (excluding noise): 0.5243
```

Task 6: Analyze Results and Cluster Statistics

- Extract cluster labels and append them back to the original dataset.
- Calculate mean statistics (e.g., average Annual Income and Spending Score) per cluster to interpret final customer segments and evaluate detected noise points.

In [7]:
```
plot_df = X.copy()
plot_df['Cluster'] = labels

plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=plot_df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='tab10',
    s=80,
    style='Cluster'
)
```

```
plt.title("DBSCAN Customer Segmentation")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.legend(title="Cluster")
plt.grid(True, alpha=0.3)
plt.show()

cluster_stats = (
    df[df['Cluster'] != -1]
    .groupby('Cluster')
    .agg(
        Customer_Count=('CustomerID', 'count'),
        Average_Age=('Age', 'mean'),
        Average_Income=('Annual Income (k$)', 'mean'),
        Average_Spending_Score=('Spending Score (1-100)', 'mean')
    )
    .round(2)
)

print("Cluster Statistics:")
display(cluster_stats)

# Analyze noise points separately
noise_customers = df[df['Cluster'] == -1]

print("\nNumber of noise customers:", len(noise_customers))
print("\nNoise customer details:")
display(noise_customers.head())

# Optional: display cluster sizes including noise
print("\nCluster sizes including noise:")
print(df['Cluster'].value_counts().sort_index())


results = []

for min_s in [4, 5, 6, 7, 8]:
    for eps in [0.2, 0.3, 0.4, 0.5, 0.6]:

        model = DBSCAN(
            eps=eps,
            min_samples=min_s
        )
        pred = model.fit_predict(X_scaled)

        n_clust = len(set(pred) - {-1})
        noise_pct = np.mean(pred == -1) * 100

        mask = pred != -1
        valid_labels = pred[mask]
        valid_data = X_scaled[mask]

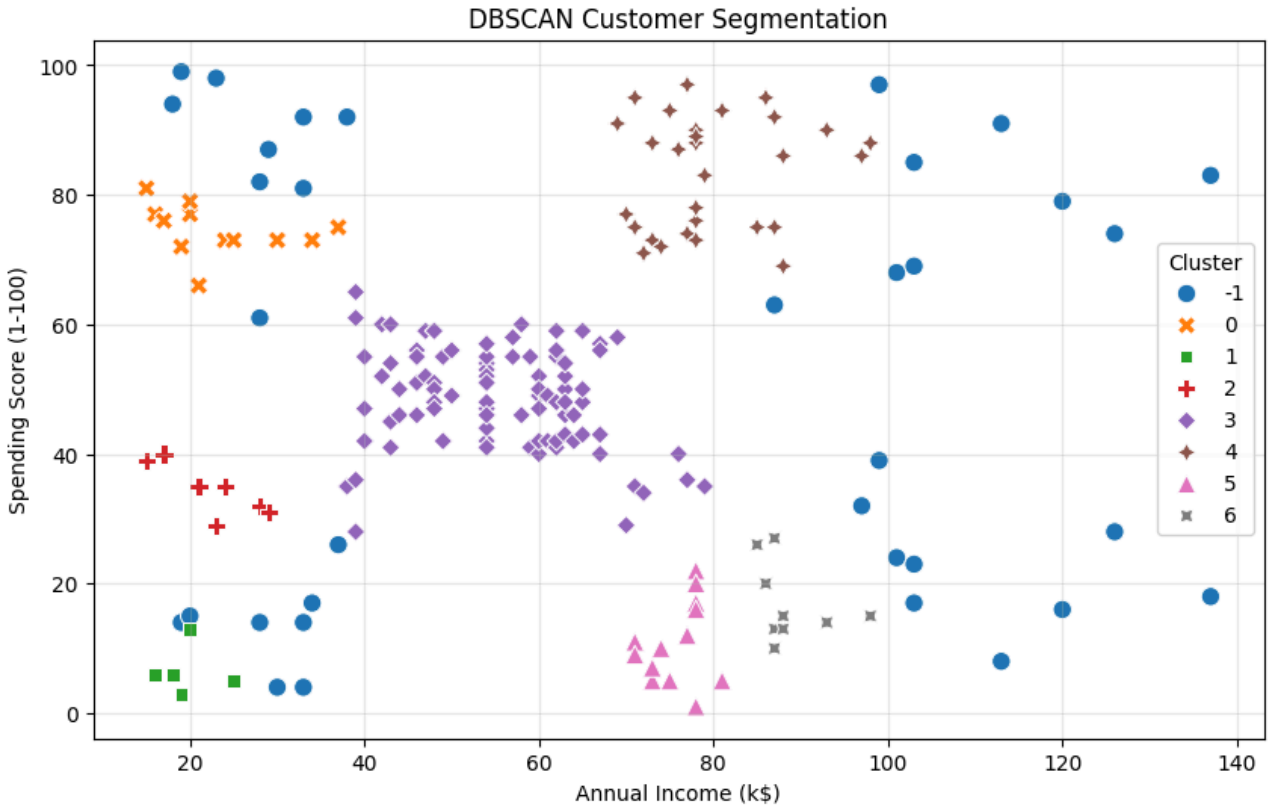
        if (
            len(np.unique(valid_labels)) >= 2
```

```
            and len(np.unique(valid_labels)) < len(valid_labels)
        ):
            sil = silhouette_score(valid_data, valid_labels)
        else:
            sil = np.nan

        results.append({
            'eps': eps,
            'min_samples': min_s,
            'clusters': n_clust,
            'noise_percentage': round(noise_pct, 2),
            'silhouette_score': sil
        })

results_df = pd.DataFrame(results)

print("DBSCAN Parameter Comparison:")
display(
    results_df.sort_values(
        by='silhouette_score',
        ascending=False,
        na_position='last'
    )
)
```



Cluster Statistics:

Cluster	Customer_Count	Average_Age	Average_Income	Average_Spending_Score
0	12	24.08	23.17	74.58
1	5	40.40	19.60	6.60
2	7	36.71	22.43	34.43
3	88	42.88	55.23	48.58
4	30	32.73	79.53	83.13
5	14	37.79	75.93	10.07
6	9	46.56	88.78	17.00

```
Number of noise customers: 35

Noise customer details:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
7	8	Female	23	18	94	-1
10	11	Male	67	19	14	-1
11	12	Female	35	19	99	-1
12	13	Female	58	20	15	-1
19	20	Female	35	23	98	-1

```
Cluster sizes including noise:
Cluster
-1    35
 0    12
 1     5
 2     7
 3    88
 4    30
 5    14
 6     9
Name: count, dtype: int64
DBSCAN Parameter Comparison:
```

	eps	min_samples	clusters	noise_percentage	silhouette_score
16	0.3	7	4	36.0	0.645159
0	0.2	4	5	36.5	0.617282
10	0.2	6	4	47.5	0.613836
15	0.2	7	4	50.0	0.609411
20	0.2	8	2	58.0	0.602745
21	0.3	8	5	36.0	0.592556
5	0.2	5	7	38.5	0.585613
22	0.4	8	5	12.5	0.548992
11	0.3	6	6	24.0	0.530244
6	0.3	5	7	17.5	0.524328
1	0.3	4	8	11.5	0.519746
17	0.4	7	4	10.0	0.494621
12	0.4	6	4	9.5	0.490015
7	0.4	5	4	7.5	0.478059
2	0.4	4	3	7.0	0.457570
13	0.5	6	2	5.5	0.401149
23	0.5	8	2	7.5	0.401006
18	0.5	7	2	6.0	0.394228
3	0.5	4	2	4.0	0.387558
8	0.5	5	2	4.0	0.387558
4	0.6	4	1	2.5	NaN
9	0.6	5	1	2.5	NaN
14	0.6	6	1	3.0	NaN
19	0.6	7	1	3.0	NaN
24	0.6	8	1	3.0	NaN

Conclusion

The experiment successfully implemented DBSCAN clustering on the Mall Customer Segmentation dataset. Feature standardization ensured balanced neighborhood distance calculations across income and spending metrics. By analyzing k-distance graphs, optimal ε and $min_samples$ parameters were selected, enabling the discovery of density-defined

customer segments and explicit identification of spatial noise points (outliers) without requiring a pre-defined cluster count.